

Long-term Time Series Forecasting with PatchTST

CS584 Machine Learning -- Final Project Proposal

Ayaan Khan | A20505209 | Spring 2026 | March 30, 2026

Problem statement

Long-term time series forecasting matters in domains like energy planning, weather prediction, and traffic management, but Transformer-based approaches have struggled with it. The core issue is how these models tokenize temporal data: feeding individual time steps as tokens means the attention mechanism operates over very long sequences, which is expensive, and each token carries almost no local context on its own. Zeng et al. (2023) showed that even a single linear layer can outperform complex Transformer architectures on standard benchmarks, raising a real question about whether Transformers belong in this space at all.

This project implements PatchTST (Nie et al., ICLR 2023), which argues that the answer is yes -- provided you rethink the tokenization. Instead of point-wise tokens, PatchTST segments each time series into subseries-level patches and treats those as tokens, much like Vision Transformers treat image patches. The goal is to reproduce the paper's main results from scratch and verify whether the reported improvements hold up under an independent implementation.

Approach

The implementation follows Option 1 (implement from scratch) with no use of the authors' source code. The model will be built in PyTorch within a Jupyter notebook. PatchTST has two main design choices that I will implement:

Patching. The input time series of length L is split into patches of length P with stride S , producing roughly L/S tokens instead of L . This cuts the self-attention cost from $O(L^2)$ to $O((L/S)^2)$, and each token now encodes a window of local temporal information rather than a single scalar. It also lets the model see much longer history without blowing up memory.

Channel independence. Each variate in a multivariate series is fed through the Transformer separately, sharing the same backbone weights. This avoids the overfitting problems that come with mixing all channels into a single token, and it scales cleanly regardless of the number of variates.

The full architecture consists of a patch embedding layer, learnable positional encodings, a standard Transformer encoder (multi-head self-attention and feed-forward layers), followed by a flatten-and-linear prediction head. I will use the supervised learning variant (PatchTST/42, look-back window $L = 336$, patch length $P = 16$, stride $S = 8$). Beyond reproducing the main results, I plan to run ablation experiments on patch length, stride, and look-back window size to verify the paper's claims about how these hyperparameters affect performance.

As a baseline, I will also implement DLinear (Zeng et al., 2023), the simple linear model that originally challenged whether Transformers are useful for LTSF. DLinear decomposes the input into trend and seasonal components via a moving average, then applies a separate linear layer to each. It takes roughly 20 lines of code and trains in seconds, but it is the model PatchTST needs to beat. Comparing my PatchTST against my own DLinear on the same data splits and preprocessing will give a fairer evaluation than citing published numbers.

Data

I will use three datasets from the standard LTSF benchmark suite, all publicly available from the ETDataset repository (github.com/zhouhaoyi/ETDataset):

Dataset	Features	Timesteps	Granularity	Split (train/val/test)
ETTh1	7	17,420	Hourly	12 / 4 / 4 months
ETTh2	7	17,420	Hourly	12 / 4 / 4 months

ETTh1 will serve as the primary development and debugging dataset due to its small size and fast iteration time. ETTh2 provides a second transformer from the same domain for comparison. Weather adds a non-ETT benchmark with more channels (21 vs. 7) to test generalization. Forecasting will be evaluated at prediction horizons of 96, 192, 336, and 720 time steps. Evaluation metrics are MSE and MAE, consistent with the original paper and prior LTSF work.

Responsibilities

This is a solo project. I am responsible for all components: data preprocessing and DataLoader construction, the from-scratch PatchTST and DLinear implementations in PyTorch, training and evaluation across all datasets and prediction horizons, ablation experiments on patch length and stride and look-back window, and the final report (8+ pages) and presentation (10-15 minutes). Training will be done on a MacBook Pro M4 Pro with 24 GB unified memory, using PyTorch's MPS backend.

References

- [1] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *Proc. ICLR*, 2023. arxiv.org/abs/2211.14730
- [2] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?" in *Proc. AAAI*, 2023. arxiv.org/abs/2205.13504
- [3] H. Zhou et al., "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI*, 2021. arxiv.org/abs/2012.07436
- [4] T. Zhou et al., "FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting," in *Proc. ICML*, 2022. arxiv.org/abs/2201.12740
- [5] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. ICLR*, 2021. arxiv.org/abs/2010.11929